ПЗ 8 Рекомендательные системы

Теоретическая часть

Рекомендации – это персонализированные предложения контента (видео, приложений и т.д.), которые создаются системами машинного обучения. Эти системы анализируют ваши интересы и поведение, чтобы находить и показывать вам то, что может вас заинтересовать. Существует два основных типа: рекомендации на главной странице (персонализированные на основе ваших общих предпочтений) и рекомендации похожих товаров (предлагают контент, аналогичный тому, который вы сейчас просматриваете). Главная цель рекомендаций – помочь пользователям открывать для себя новый релевантный контент в огромном массиве данных, который они могли бы не найти с помощью простого поиска.

Рекомендательная система работает с **предметами** (документы, видео, приложения), которые нужно рекомендовать, и **запросами** (контекст), которые включают информацию о пользователе, его историю взаимодействий и дополнительные данные (время, устройство). Ключевым механизмом является **встраивание** – преобразование этих дискретных запросов и предметов в векторы в общем пространстве, что позволяет системе находить и рекомендовать наиболее релевантные для пользователя предметы на основе их семантической близости.

Рекомендательная система обычно состоит из трех этапов: **генерация кандидатов**, которая быстро отбирает из миллионов элементов небольшой пул (сотни или тысячи) потенциально релевантных позиций; **ранжирование**, где более точная модель оценивает и сортирует этих кандидатов, чтобы выбрать лучшие (около 10); и **реранжирование**, которое применяет бизнес-правила для финальной корректировки списка, обеспечивая свежесть, разнообразие и соответствие дополнительным ограничениям.

Генерация кандидатов - это первоначальный этап рекомендательной системы, который формирует узкий круг потенциально релевантных элементов из обширного каталога. Для этого используются два основных подхода: фильтрация по контенту, которая предлагает элементы, похожие на те, что пользователь уже предпочитал, и совместная фильтрация, которая рекомендует элементы, понравившиеся схожим с ним пользователям. Оба метода работают в общем пространстве низкой размерности, называемом пространством вложений, где семантически близкие объекты (запросы и элементы) оказываются рядом.

Ключевым механизмом для поиска кандидатов является мера сходства, которая вычисляет близость между векторами запроса и элементов. Наиболее распространенными мерами являются косинусная близость, скалярное произведение и евклидово расстояние. Выбор меры напрямую влияет на результат: например, скалярное произведение чувствительно к норме вектора, что может усилить влияние популярных элементов с большими нормами

вложений, тогда как косинусная близость фокусируется исключительно на направлении векторов, игнорируя их длину.

На практике выбор меры сходства является компромиссом. Если цель – учитывать популярность контента, предпочтительнее скалярное произведение. Однако это может привести к доминированию в рекомендациях самых популярных позиций. Чтобы управлять этим влиянием, можно использовать модифицированные метрики. Кроме того, для редких элементов с нестабильными вложениями важны корректная инициализация и регуляризация, чтобы избежать нерелевантных рекомендаций.

Фильтрация по контенту – это метод рекомендаций, который предлагает пользователю элементы, похожие на те, что он уже предпочитал, основываясь на сходстве характеристик этих элементов. Пользователь и каждый элемент представляются в виде векторов в одном пространстве признаков, таких как категория, издатель или жанр. Например, если пользователь установил несколько образовательных приложений от одного издателя, его вектор будет отражать активность этих признаков.

Мера сходства, такая как скалярное произведение, используется для вычисления релевантности каждого элемента пользователю. В случае бинарных векторов скалярное произведение просто подсчитывает количество общих активных признаков между вектором пользователя и вектором элемента. Чем больше общих признаков, тем выше оценка сходства и тем вероятнее, что элемент будет рекомендован.

Этот подход полностью персонализирован для конкретного пользователя, так как не использует данные о других пользователях. Система просто ранжирует все элементы-кандидаты по их сходству с профилем пользователя и выбирает для рекомендации те, что имеют наибольшее количество общих с ним характеристик.

Совместная фильтрация — это метод рекомендаций, который преодолевает ограничения контентных методов, одновременно используя сходства между пользователями и элементами. Вместо ручного создания признаков она автоматически обучает векторные представления (эмбеддинги) как для пользователей, так и для элементов в общем пространстве.

Основной принцип заключается в том, что пользователи и элементы представляются в одном и том же низкоразмерном векторном пространстве. Скалярное произведение вектора пользователя и вектора элемента должно приближенно отражать вероятность того, что пользователю понравится этот элемент. Например, если два пользователя имеют схожие вкусы, их векторы будут близки в пространстве, и система сможет рекомендовать одному пользователю элементы, которые понравились другому, даже если эти элементы сильно различаются по своим контентным характеристикам.

Размерность пространства имеет ключевое значение. Как показано на примере с фильмами, одномерного пространства (например, ось "детское-взрослое") может быть недостаточно для точного описания предпочтений. Добавление второй размерности (например, ось "блокбастер-артхаус")

позволяет лучше объяснить разнообразные вкусы пользователей и более точно предсказать их интересы.

Матричная факторизация – это модель для построения рекомендаций, которая аппроксимирует разреженную матрицу взаимодействий "пользователь-элемент" через произведение двух низкоразмерных матриц: матрицы эмбеддингов пользователей и матрицы эмбеддингов элементов. Скалярное произведение вектора пользователя и вектора элемента стремится приблизиться к известному значению взаимодействия (например, рейтингу). Этот подход эффективно выявляет скрытые структуры данных, значительно сокращая объем параметров по сравнению с хранением полной матрицы.

Ключевой задачей является выбор целевой функции. Наивная минимизация ошибки только по наблюдаемым взаимодействиям приводит к тривиальному решению, когда модель рекомендует всё подряд. Минимизация ошибки по всем возможным парам (включая нулевые взаимодействия) через SVD также неэффективна из-за разреженности данных. **Взвешенная матричная факторизация** предлагает компромисс, вводя гиперпараметр для баланса между ошибками на наблюдаемых и ненаблюдаемых парах, что позволяет избежать переобучения и улучшить обобщающую способность.

Для минимизации целевой функции применяются два основных алгоритма. Стохастический градиентный спуск (SGD) более гибкий и позволяет использовать различные функции потерь, но может сходиться медленнее и требует специальных методов обработки ненаблюдаемых записей. **Взвешенный метод попеременных наименьших квадратов (WALS)**, специализированный для этой задачи, попеременно оптимизирует матрицы пользователей и элементов, сходится быстрее и эффективнее работает с непросмотренными взаимодействиями, хотя и заложен в рамки квадратичной функции потерь.

Получив эмбеддинг пользователя (запроса), система решает задачу поиска ближайших соседей в пространстве эмбеддингов, чтобы найти наиболее релевантные элементы. Для этого вычисляется мера сходства (например, косинусная близость) между вектором пользователя и всеми векторами элементов, после чего выбираются `k` элементов с наивысшими оценками.

Прямой перебор всех кандидатов (исчерпывающий поиск) вычислительно дорог для больших каталогов. Для оптимизации используются стратегии вроде предварительного вычисления топ-рекомендаций для статических запросов или применение приближенных методов поиска (например, ScaNN), которые жертвуют точностью ради скорости, что критически важно в реальных системах с миллиардами элементов.

Этап оценки (scoring) следует за генерацией кандидатов и отвечает за точное ранжирование узкого списка потенциальных элементов. Для этого используется отдельная, часто более сложная модель, которая анализирует объединенный пул кандидатов от разных источников (например, похожие элементы, тренды, соцграф). Эта модель учитывает широкий контекст:

признаки пользователя, элемента и запроса, чтобы спрогнозировать целевую метрику, такую как вероятность просмотра или вовлечения.

Критически важным является выбор целевой функции для модели. Прямая оптимизация под простые метрики (например, кликабельность) может привести к нежелательным последствиям, таким как продвижение кликбейта. Более сложные цели, такие как общее время просмотра в сессии или разнообразие выдачи, часто приводят к более качественным и полезным для пользователя рекомендациям.

Ключевой проблемой является позиционное смещение: пользователи чаще кликают на элементы вверху списка, независимо от их качества. Чтобы избежать этого, система обучает модель, которая не зависит от позиции, и ранжирует кандидатов так, как если бы все они находились на самой заметной позиции экрана, что обеспечивает более объективную оценку релевантности.

Этап повторного ранжирования – это финальный шаг в системе рекомендаций, где отсортированный список кандидатов корректируется с учетом дополнительных бизнес-правил и требований к качеству. Это может включать фильтрацию нежелательного контента (например, кликбейта) или ручную корректировку оценок на основе таких факторов, как свежесть материала или его длительность.

Ключевыми целями повторного ранжирования являются обеспечение свежести и разнообразия выдачи. Чтобы поддерживать актуальность, система использует частое переобучение на новых данных, добавляет возраст контента как признак и применяет модели, способные работать с новыми пользователями и элементами. Для борьбы с однообразием используются множественные генераторы кандидатов и ранкеры, а также вводятся правила, продвигающие контент из разных категорий.

Важным аспектом является обеспечение справедливости рекомендаций. Система должна избегать усиления предубеждений, заложенных в данных. Для этого необходимы всесторонние данные для обучения, мониторинг метрик по разным пользовательским группам и, в некоторых случаях, создание отдельных моделей для недостаточно представленных аудиторий.

Ключевые аспекты построения рекомендательных систем:

1. Назначение и компоненты системы

1.1 Помочь пользователям открывать релевантный контент в огромном массиве данных, который они могли бы не найти самостоятельно.

1.2 Система состоит из трех этапов:

Генерация кандидатов (быстрый отбор небольшого подмножества потенциально релевантных элементов из всего каталога).

Ранжирование (Скоринг) (точная оценка и сортировка кандидатов с помощью сложной модели для выбора лучших.

Повторное ранжирование (финальная корректировка списка с учетом бизнес-правил, свежести, разнообразия и справедливости).

2. Эмбеддинги и основные подходы

Эмбеддинги - это векторные представления пользователей и элементов в общем пространстве меньшей размерности, которые захватывает их скрытые характеристики и семантическое сходство.

Подходы:

Фильтрация по контенту - рекомендует элементы, похожие на те, что пользователь уже предпочитал, на основе сходства их атрибутов.

Совместная фильтрация - рекомендует элементы на основе сходства с другими пользователями, автоматически обучая эмбеддинги.

3. Модели и их эволюция

Матричная факторизация - классическая модель, которая аппроксимирует матрицу взаимодействий через произведение эмбеддингов пользователей и элементов. Эффективна для выявления скрытых структур.

Глубокие нейронные сети (DNN) - преодолевают ограничения матричной факторизации, позволяя использовать более богатые признаки (например, контекст, возраст контента) и моделировать более сложные нелинейные зависимости.

4. Процесс построения системы

1. **Поиск** (после получения эмбеддинга запроса система ищет ближайших соседей в пространстве эмбеддингов элементов, используя эффективные методы (например, приближенный поиск).
2. **Оценка** (объединенные кандидаты от разных источников оцениваются и ранжируются отдельной моделью, которая учитывает широкий контекст для максимизации целевой метрики).
3. **Переранжирование** (финальный список корректируется для обеспечения свежести, разнообразия, справедливости и соответствия дополнительным правилам).

Методические рекомендации к выполнению задания:

Имеются данные о просмотрах фильмов и соответствующих оценок по каждому фильму (https://files.grouplens.org/datasets/movielens/ml-100k/u.data). Требуется построить вариант рекомендательной системы, которая будет работать по принципу: «Люди, которые смотрели те же фильмы, что и ты, тоже смотрели...»

1 вариант реализации - пример:

```
# Самый простой способ получения данных:
import pandas as pd
url = "https://files.grouplens.org/datasets/movielens/ml-100k/u.data"
columns = ['user_id', 'movie_id', 'rating', 'timestamp']
ratings = pd.read_csv(url, sep='\t', names=columns)
```

```
import pandas as pd
import numpy as np
```

```
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.decomposition import TruncatedSVD

# Создаем демо-датасет (рейтинги фильмов от пользователей). Здесь используем данные выше.
data = {
    'user_id': [1, 1, 1, 2, 2, 2, 3, 3, 3, 4, 4, 4, 5, 5, 5],
    'movie_id': [1, 2, 3, 1, 2, 4, 1, 3, 5, 2, 4, 5, 3, 4, 5],
    'rating': [5, 4, 3, 4, 5, 2, 3, 4, 5, 4, 3, 2, 5, 4, 3]
}

df = pd.DataFrame(data)
print("Исходные данные:")
print(df)

# Создаем матрицу "пользователь-фильм"
user_movie_matrix = df.pivot_table(
    index='user_id',
    columns='movie_id',
    values='rating',
    fill_value=0
)

print("\nМатрица пользователь-фильм:")
print(user_movie_matrix)

# Вычисляем косинусное сходство между фильмами
movie_similarity = cosine_similarity(user_movie_matrix.T)
movie_similarity_df = pd.DataFrame(
    movie_similarity,
    index=user_movie_matrix.columns,
    columns=user_movie_matrix.columns
)

print("\nСходство между фильмами:")
print(movie_similarity_df)

# Функция для рекомендаций
def recommend_movies(movie_id, n_recommendations=3):
    """Рекомендует фильмы на основе сходства"""
    similar_movies = movie_similarity_df[movie_id].sort_values(ascending=False)
    # Исключаем сам фильм из рекомендаций
    similar_movies = similar_movies[similar_movies.index != movie_id]
    return similar_movies.head(n_recommendations)
```

```
# Тестируем рекомендации
print("\nРекомендации для фильма 1:")
print(recommend_movies(1))

print("\nРекомендации для фильма 3:")
print(recommend_movies(3))
```

2 вариант реализации - продвинутая версия:

```
# Используем SVD для уменьшения размерности
svd = TruncatedSVD(n_components=2)
matrix_reduced = svd.fit_transform(user_movie_matrix)

# Преобразуем обратно в матрицу сходства
matrix_approx = svd.inverse_transform(matrix_reduced)
matrix_approx_df = pd.DataFrame(
    matrix_approx,
    index=user_movie_matrix.index,
    columns=user_movie_matrix.columns
)

print("Приближенная матрица после SVD:")
print(matrix_approx_df.round(2))

def recommend_for_user(user_id, n_recommendations=2):
    """Рекомендует фильмы для конкретного пользователя"""
    user_ratings = matrix_approx_df.loc[user_id]
    # Ищем фильмы с высоким рейтингом, которые пользователь еще не
смотрел
    watched_movies = user_movie_matrix.loc[user_id]
    unwatched_movies = user_ratings[watched_movies == 0]
    return
unwatched_movies.sort_values(ascending=False).head(n_recommendations)

print("\nРекомендации для пользователя 1:")
print(recommend_for_user(1))
```

Вопросы для самоподготовки

1. Каковы две основные цели рекомендательных систем?
2. Назовите три основных этапа работы рекомендательной системы и кратко охарактеризуйте каждый.
3. В чем ключевое различие между контентной фильтрацией и совместной фильтрацией?
4. Что такое эмбеддинги и какую роль они играют в рекомендательных системах?

5. Какие три основные меры сходства используются при поиске кандидатов и чем они отличаются?
6. Какие проблемы решает взвешенная матричная факторизация по сравнению с наивными подходами?
7. Почему на этапе оценки (scoring) используется отдельная модель вместо генератора кандидатов?
8. Что такое позиционное смещение и как с ним борются?
9. Какие цели преследует этап повторного ранжирования?
10. Какие преимущества DNN имеют перед матричной факторизацией?